The Journal of Open Source Software

itwinai: A Python Toolkit for Scalable Scientific Machine Learning on HPC Systems

Matteo Bunino[1], **Jarl Sondre Sæther**[1], **Linus Maximilian Eickhoff**[1], **Anna Elisa Lappe**[1], **Kalliopi Tsolaki**[1], **Killian Verder**[1], **Henry Mutegeki**[1], **Roman Machacek**[1], **Maria Girone**[1], **Oleksandr Krochak**[2], **Mario Rüttgers**[2,3], **Rakesh Sarma**[2], and **Andreas Lintermann**[2]

1 European Organization for Nuclear Research (CERN), Espl. des Particules 1, 1217 Genève, Switzerland **2** Simulation and Data Lab Fluids & Solids Engineering, Jülich Supercomputing Center, Forschungszentrum Jülich, Germany **3** Data-Driven Fluid Engineering (DDFE) Laboratory, Inha University, Incheon, Republic of Korea

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 02 September 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

The integration of Artificial Intelligence (AI) into scientific research has expanded significantly over the past decade, driven by the availability of large-scale datasets and specialized hardware accelerators. Graphics Processing Units (GPUs) have become essential for AI workloads, with modern High-Performance Computing (HPC) facilities housing thousands of accelerators: the JUWELS Booster system (Krause, 2019) at the Jülich Supercomputing Center (JSC), Forschungszentrum Jülich, operates over 3,700 NVIDIA A100 GPUs, while Finland's LUMI supercomputer provides access to more than 11,000 AMD MI250X GPUs, and the upcoming first European Exascale system JUPITER hosted at JSC, expanding capabilities up to 24,000 NVIDIA GH200 GPUs.

This growth has enabled development of numerous domain-specific AI solutions across scientific fields, offering the potential for faster experimentation and shorter feedback cycles. However, many researchers face significant barriers when deploying AI workflows on HPC systems due to the complexity of, e.g., running distributed training and Hyperparameter Optimization (HPO) at scale, or in utilizing accelerator technologies efficiently. The heterogeneous nature of HPC systems forces scientists to focus on low-level implementation details rather than on their core research.

To address these challenges, we present `itwinai`, a Python library designed to bridge the gap between domain sciences, AI, and HPC. The library provides a standardized interface, enabling seamless scaling from laptops to supercomputers. Through its modular architecture, `itwinai` significantly lowers the entry barrier for domain specialists while ensuring optimal HPC resource utilization.

Statement of need

The integration of diverse ML tools and libraries requires significant integration effort on the part of the scientific users, especially with HPC systems. `itwinai` is a Python library that streamlines the integration of AI-powered scientific workflows into HPC infrastructure through ready-to-use components. The library addresses three core scientific Machine Learning (ML) needs:

Distributed training with uniform interfaces across PyTorch-DDP (Li et al., 2020), DeepSpeed (Rasley et al., 2020), and Horovod (Sergeev & Del Balso, 2018) backends.

Scalable HPO via Ray Tune (Liaw et al., 2018) supporting various search algorithms.

Comprehensive performance analysis including profiling, scaling and parallel efficiency metrics, and bottleneck identification.

Built with a modular architecture, `itwinai` enables component integration and plugin-based extensions for independent development of scientific use cases. Developed as part of the interTwin[1] project to support Digital Twin applications across physics and environmental sciences, the library has proven versatile for general AI applications. By consolidating diverse functionalities into a single framework, `itwinai` significantly lowers the barrier to HPC adoption and empowers the scientific community to scale AI workloads efficiently.

Package features

The main features offered by the `itwinai` library are:

Configuration for reproducible AI workloads: `itwinai` employs a declarative, YAML-based configuration system that separates experimental parameters from implementation code. Configurations are hierarchical, composable, and Command Line Interface (CLI)-overrideable. This design ensures reproducible and portable executions across environments (workstation, cloud, HPC systems) with minimal code changes.

Distributed training and inference: `itwinai` supports multiple distributed ML training frameworks, including PyTorch-DDP, DeepSpeed, Horovod, and Ray (Moritz et al., 2018). The underlying implementation of these frameworks is hidden from the user. The parallelization strategy can simply be changed by setting a flag, to, e.g., investigate potential performance gains.

Hyperparameter optimization: Leveraging Ray, `itwinai` allows to systematically improve model performance by searching the hyperparameter space. Ray enables scaling HPO across HPC systems by (i) assigning multiple workers to a single trial (e.g., data-parallel training) and (ii) running many trials concurrently to exploit massive parallelism. It also provides robust scheduling and fine-grained resource management for large, distributed studies (Figure 1).

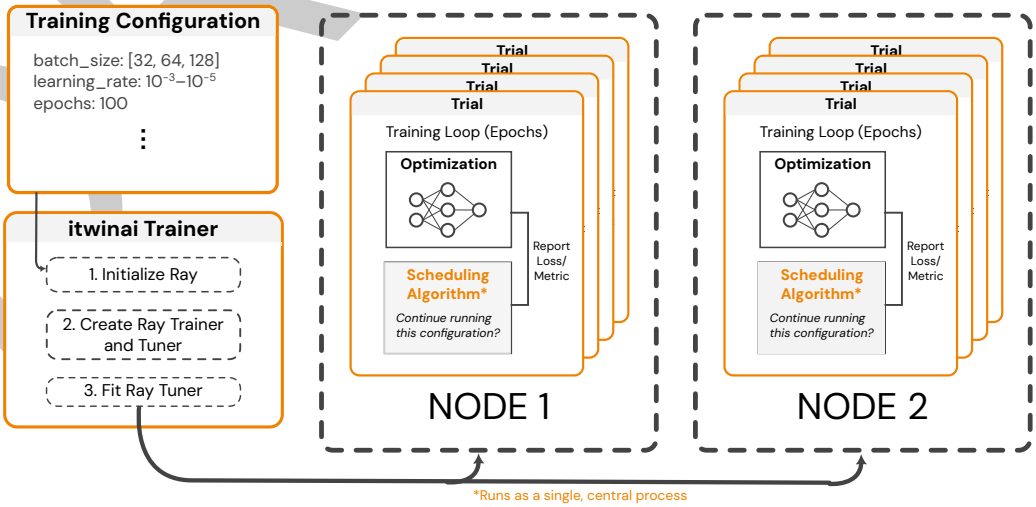

Figure 1: Conceptual representation of an HPO workflow in itwinai.

Profilers: `itwinai` integrates with multiple profilers, such as the `py-spy` profiler (Frederickson, 2018) and the PyTorch Profiler, and also logs metrics about training time, GPU utilization,

[1] interTwin project: intertwin.eu (Accessed on 2025-08-14).

and GPU power consumption.

ML logs tracking: `itwinai` integrates with existing ML logging frameworks, such as TensorBoard (TensorBoard Contributors, 2025), Mlflow (Zaharia et al., 2018), Weights&Biases (wandb Contributors, 2025), and yProvML (Padovani & Fiore, 2025) logger, and provides a unified interface across all of them through a thin abstraction layer.

Offloading to HPC systems and cloud: To benefit from both cloud and HPC, interLink (Ciangottini et al., 2025) is used, which is a lightweight component to enable seamless offloading of compute-intensive jobs from cloud to HPC, performing an automatic translation from Kubernetes pods to SLURM jobs.

Continuous integration and deployment `itwinai` includes extensive tests (library and use cases). A Dagger pipeline[2] builds containers on release, runs smoke tests on GitHub Actions (Azure runners: 4 CPUs, 16 GB)[3], offloads distributed tests to HPC systems via interLink, and publishes on success.

Use-case integrations

There is a wide range of scientific use cases currently integrated with `itwinai` via its plug-in architecture. Earth-observation plugins cover hydrological forecasting, drought prediction, and climate/remote-sensing pipelines; physics plugins include high-energy physics, radio astronomy, lattice quantum chromodynamics (QCD), and gravitational-wave/glitch analysis. Packaging these as `itwinai` plugins enables reproducible, shareable workflows that run consistently on hardware ranging from personal computers to HPC systems. The full list of `itwinai` plugins can be found at this link.

Performance

`itwinai` provides tools to assess scalability and diagnose bottlenecks, enabling efficient and accountable use of HPC resources. Two complementary components are provided: scalability report generation and profiling.

Scalability report

For data-parallel training, adding more workers improves throughput, but as all-reduce communication costs grow, communication overhead eventually dominates, causing scaling to level-off or even decline. The report characterizes this trade-off across GPUs/nodes and backends, reporting wall-clock epoch time, relative speedup (Figure 2), GPU utilization (0–100%), energy (Wh), and compute-versus-other time, including collective communication and memory operations (Figure 3). Considered jointly, these metrics identify the most efficient configuration and distribution strategy, rather than relying on a single indicator. Figure 2 and Figure 3 show the scalability of the physics use case from INFN[4] targeting gravitational-wave analysis at the Virgo[5] interferometer (Tsolaki et al., 2025) (Sæther et al., 2025).

[2]Dagger: dagger.io (Accessed on 2025-08-14).

[3]GitHub hosted runners define the type of machine that will process a job in your workflow. Find more here (Accessed on 2025-08-14).

[4]Istituto Nazionale di Fisica Nucleare infn.it (Accessed on 2025-08-14).

[5]Virgo Collaboration www.virgo-gw.eu (Accessed on 2025-08-14).

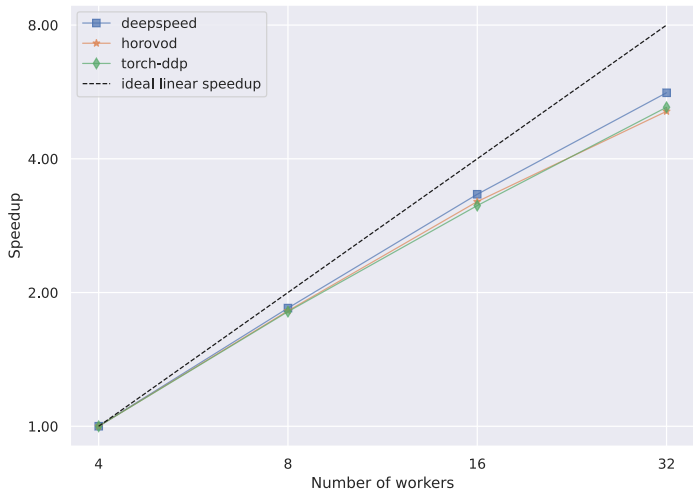

Figure 2: Relative speedup of average epoch time vs. number of workers for the Virgo use case.

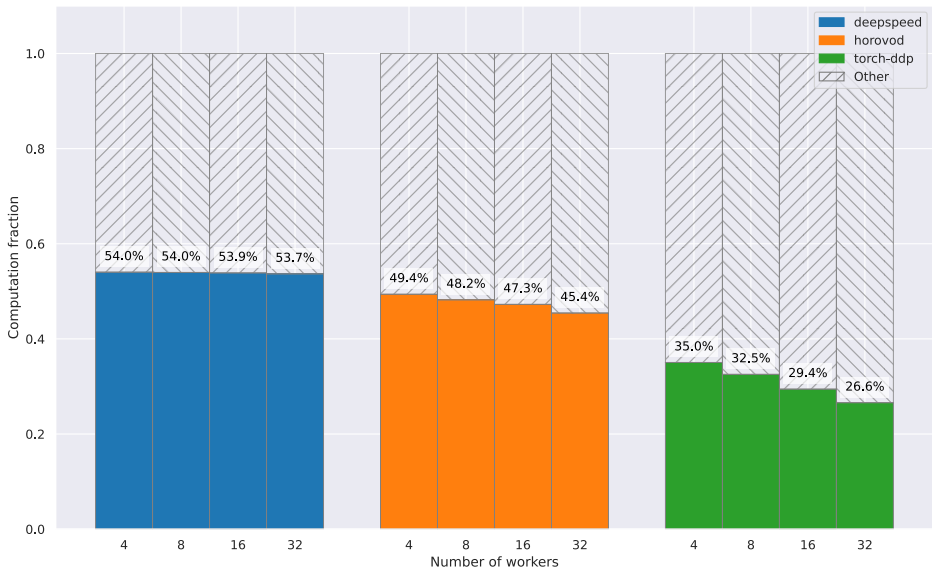

Figure 3: Proportion of time spent on computation versus other operations, such as collective communication, in the Virgo use case, broken down by number of workers and distributed framework.

Addressing bottlenecks via profiling

To explain why performance degrades, `itwinai` integrates low-overhead, sample-based profiling (e.g., py-spy (Frederickson, 2018)) and summarizes flame-graph data into actionable hotspots (e.g., data loading and I/O, kernel execution, host–device transfer, communication). These summaries guide targeted remedies such as adjusting batch size, data-loader parallelism, gradient accumulation, or backend/collective settings.

Outlook and future developments

`itwinai` provides ready-to-use ML tools that are applicable across a wide range of scientific applications. The development of the library is continued through other follow-up projects, such as ODISSEE[6] and RI-SCALE[7]. The future developments include the integration of new scientific use cases, exploring additional parallelism approaches, integrating advanced user interfaces, and adding other EuroHPC systems and performance optimization features.

Acknowledgements

This work has been funded by the European Commission in the context of the interTwin project, with Grant Agreement Number 101058386. In interTwin, `itwinai` has been actively developed on the HPC systems at JSC, such as on the HDF-ML and JUWELS Booster systems, and using EuroHPC resources, such as on the Vega HPC system. `itwinai` is an open-source Python library primarily developed by CERN, in collaboration with Forschungszentrum Jülich (FZJ).

References

Ciangottini, D., Bianchini, G., & Spiga, D. (2025). interLink. In *GitHub repository*. GitHub. https://github.com/interLink-hq/interLink

Frederickson, B. (2018). py-spy: Sampling profiler for Python programs. In *GitHub repository*. GitHub. https://github.com/benfred/py-spy

Krause, D. (2019). JUWELS: Modular tier-0/1 supercomputer at the Jülich Supercomputing Centre. *J. Of Large-Scale Research Facilities*, *5*(A135). https://doi.org/10.17815/jlsrf-5-171

Li, S., Zhao, Y., Varma, R., Salpekar, O., Noordhuis, P., Li, T., Paszke, A., Smith, J., Vaughan, B., Damania, P., & others. (2020). Pytorch Distributed: Experiences on accelerating data parallel training. *arXiv:2006.15704*. https://doi.org/10.48550/arXiv.2006.15704

Liaw, R., Liang, E., Nishihara, R., Moritz, P., Gonzalez, J. E., & Stoica, I. (2018). *Tune: A research platform for distributed model selection and training*. arXiv. https://arxiv.org/abs/1807.05118

Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., & Stoica, I. (2018). *Ray: A distributed framework for emerging AI applications*. https://arxiv.org/abs/1712.05889

Padovani, G., & Fiore, S. (2025). yProvML. In *GitHub repository*. GitHub. https://github.com/HPCI-Lab/yProvML

Rasley, J., Rajbhandari, S., Ruwase, O., & He, Y. (2020). Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. *Proc. 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 3505–3506. https://doi.org/10.1145/3394486.3406703

Sæther, J. S., Bunino, M., & Eickhoff, L. M. (2025). *Scalability analysis of GlitchFlow with itwinai*. Zenodo. https://zenodo.org/records/16882390

Sergeev, A., & Del Balso, M. (2018). Horovod: fast and easy distributed deep learning in TensorFlow. *arXiv:1802.05799*. https://doi.org/10.48550/arXiv.1802.05799

[6]Online Data Intensive Solutions for Science in the Exabytes Era (ODISSEE): odissee-project.eu (Accessed on 2025-08-14).

[7]RI-SCALE project: riscale.eu (Accessed on 2025-08-14).

DRAFT

TensorBoard Contributors. (2025). TensorBoard. In *GitHub repository*. GitHub. https://github.com/tensorflow/tensorboard

Tsolaki, K., Vallecorsa, S., Vallero, S., Asprea, L., Sarandrea, F., Komijani, J., Ray, G. S., Pidopryhora, Y., & Campos, I. (2025). *interTwin D4.6 final architecture design of the DTs capabilities for high energy physics, radio astronomy and gravitational-wave astrophysics* (1 Under EC Review). Zenodo. https://doi.org/10.5281/zenodo.15120028

wandb Contributors. (2025). Weights & Biases. In *GitHub repository*. GitHub. https://github.com/wandb/wandb

Zaharia, M. A., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Konwinski, A., Murching, S., Nykodym, T., Ogilvie, P., Parkhe, M., Xie, F., & Zumar, C. (2018). Accelerating the Machine Learning Lifecycle with MLflow. *IEEE Data Eng. Bull.*, *41*, 39–45. https://api.semanticscholar.org/CorpusID:83459546

DRAFT